Dockerized Healthcare Python Flask Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	**ANIMESH BOKIL**	**EN22CS301125**
02	**ANSHIKA VYAS**	**EN22CS301154**
03	**ARYAN SINGH**	**EN22CS301215**
04	**ASHLESHA VERMA**	**EN22CS301219**
05	**AVANI RAWAL**	**EN22CS301238**

Group Name:10D2

Project Number:DO-22

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2025-2026

MEDICAPS UNIVERSITY

Datagami
"Lead Digital Technology"

TABLE OF CONTENTS

S.NO.	TOPICS	
1	INTRODUCTION	1.1 SCOPE OF THE DOCUMENT
		1.2 INTENDED AUDIENCE
		1.3 SYSTEM OVERVIEW
2	SYSTEM DESIGN	2.1 APPLICATION DESIGN
		2.2 PROCESS FLOW
		2.3 INFORMATION FLOW
		2.4 COMPONENTS DESIGN
		2.5 KEY DESIGN CONSIDERATIONS
		2.6 API CATALOGUE
3.	DATA DESIGN	3.1 DATA MODEL
		3.2 DATA ACCESS MECHANISM
		3.3 DATA RETENTION POLICIES
		3.4 DATA MIGRATION
4.	INTERFACES	
5.	STATE AND SESSION MANAGEMENT	
6.	CACHING	
7.	FUNCTIONAL REQUIREMENTS	7.1 AUTOMATED BUILD AND TESTING PROCESS
		7.2 APPLICATION CONTAINERIZATION USING DOCKER
		7.3 DEPLOYMENT ON AWS EC2
		7.4 MONITORING AND LOGGING
8.	NON-FUNCTIONAL REQUIREMENTS	8.1 SECURITY

		8.2 PERFORMANCE
		8.3 SCALABILITY
		8.4 RELIABILITY
9.	**REFERENCES**	

1. INTRODUCTION

The healthcare industry increasingly relies on digital systems to manage patient data, appointments, and medical records efficiently. However, deploying such applications across different environments often leads to inconsistencies, dependency issues, and scalability challenges. Traditional deployment approaches lack portability and are prone to configuration errors.To address these challenges, this project focuses on containerizing a Healthcare Service built using Python Flask using Docker. Containerization ensures that the application, along with its dependencies, runs consistently across development, testing, and production environments.The system leverages multi-stage Docker builds, which help reduce the final image size by separating the build environment from the runtime environment. This not only improves performance but also enhances security by minimizing unnecessary packages in the final image.

Additionally, Docker Compose is used to orchestrate multiple services, including the Flask backend and the database. This allows developers to start the entire application stack with a single command, improving productivity and collaboration.The primary objective of this project is to create a portable, scalable, secure, and efficient deployment architecture for a healthcare application using modern DevOps practices.

1.1 SCOPE OF THE DOCUMENT

This document presents the High-Level Design (HLD) of the containerized Healthcare Service.It provides a comprehensive overview of the system architecture, design principles, and workflow involved in building and deploying the application.

The document focuses on:

- Containerization strategy using Docker
- Multi-stage Dockerfile design for optimization
- Service orchestration using Docker Compose
- Data management and interaction between services
- Functional and non-functional requirements

It does not include low-level implementation details such as exact code or configurations but provides enough information to understand the overall system structure.

1.2 INTENDED AUDIENCE

This document is intended for multiple stakeholders involved in the project:

- **Developers**: To understand how the application is structured and how it is containerized
- **DevOps Engineers**: To manage Docker images, containers, and orchestration
- **System Administrators**: To deploy, monitor, and maintain the application
- **Project Evaluators**: To assess the system design, scalability, and feasibility

1.3 SYSTEM OVERVIEW

The system is a Flask-based Healthcare Service designed to manage healthcare-related operations such as patient data, appointments, and medical records.

The application is deployed using a containerized architecture consisting of:

- A Flask backend service
- A database service (such as PostgreSQL or MySQL)
- Docker containers to encapsulate each service
- Docker Compose to manage multi-container deployment

When the system is started, Docker Compose launches all required services, establishesnetworking between them, and ensures that the application is ready to handle user requests.

This approach ensures:

- Consistency across environments
- Faster deployment
- Easy scalability
- Improved system reliability

2. SYSTEM DESIGN

The system design is based on a containerized architecture, where each component runs in an isolated environment. This improves modularity, maintainability, and scalability.

2.1 APPLICATION DESIGN

The The application follows a modular design consisting of multiple layers:
1. Presentation Layer
Handles incoming HTTP requests from users or client applications.
2. Application Layer (Flask Backend)
- Processes business logic
- Validates input data
- Handles API routing

3. Data Layer
- Stores and retrieves healthcare data
- Managed by a relational database

4. Containerization Layer
- Docker packages the application and its dependencies
- Multi-stage builds optimize the image

5. Orchestration Layer
- Docker Compose manages multiple containers
- Handles networking and service dependencies

This layered architecture ensures separation of concerns and improves maintainability.

2.2 PROCESS FLOW

The process flow of the system is as follows:

1. The developer writes the Flask application code
2. A Dockerfile is created to define the application environment
3. Multi-stage build compiles dependencies and creates a lightweight image
4. Docker Compose is used to start the application and database containers
5. The backend service connects to the database using environment variables
6. Users send API requests to the application
7. The backend processes the request and interacts with the database
8. The response is returned to the user

This workflow ensures efficient development and deployment.

2.3 INFORMATION FLOW

The flow of information within the system is structured as follows:

1. A client sends an HTTP request to the Flask API
2. The request is received by the backend container
3. The backend validates and processes the request
4. Data is fetched or updated in the database container
5. The database returns the result to the backend
6. The backend formats the response
7. The response is sent back to the client

This structured flow ensures smooth communication between components.

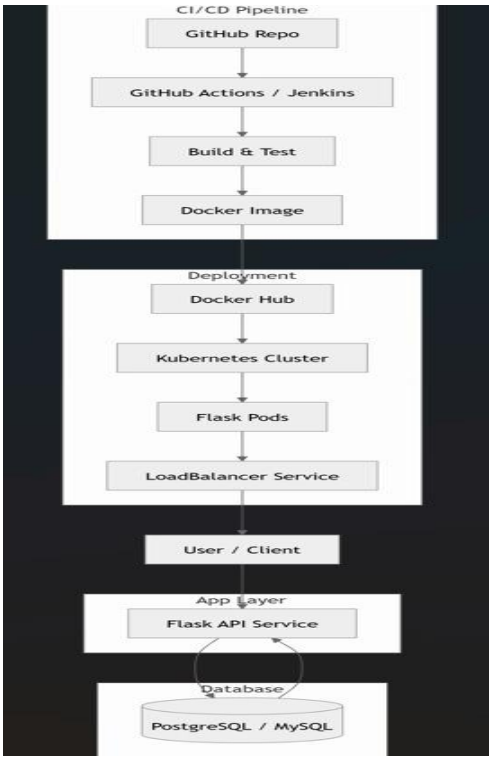

Figure 1 :High-Level System Architecture

2.4 COMPONENTS DESIGN

The system consists of the following key components:

1. Flask Application Container

- Hosts the backend application
- Handles API requests

2. Database Container

- Stores application data
- Ensures data persistence

3. Docker Engine

- Builds and runs containers
- Manages container lifecycle

4. Docker Compose

- Orchestrates multiple containers
- Defines services in a YAML file

Each component is independent yet interconnected, ensuring modularity.

2.5 KEY DESIGN CONSIDERATIONS

Several important factors were considered during system design:

- **Security**: Use of minimal base images and environment variables
- **Performance**: Optimized Docker layers and reduced image size
- **Scalability**: Ability to scale containers horizontally
- **Portability**: Runs on any system with Docker installed
- **Maintainability**: Modular structure for easy updates

2.6 API CATALOGUE

The application exposes REST APIs such as:

- GET /patients – Retrieve patient details
- POST /patients – Add a new patient
- GET /appointments – Fetch appointment data
- POST /appointments – Schedule appointments

These APIs enable communication between the frontend and backend.

3. DATA DESIGN

The data design focuses on managing structured healthcare information efficiently and securely.

3.1 DATA MODEL

The system includes entities such as:

- Patient
- Doctor
- Appointment
- Medical Records

These entities are stored in relational tables with proper relationships.

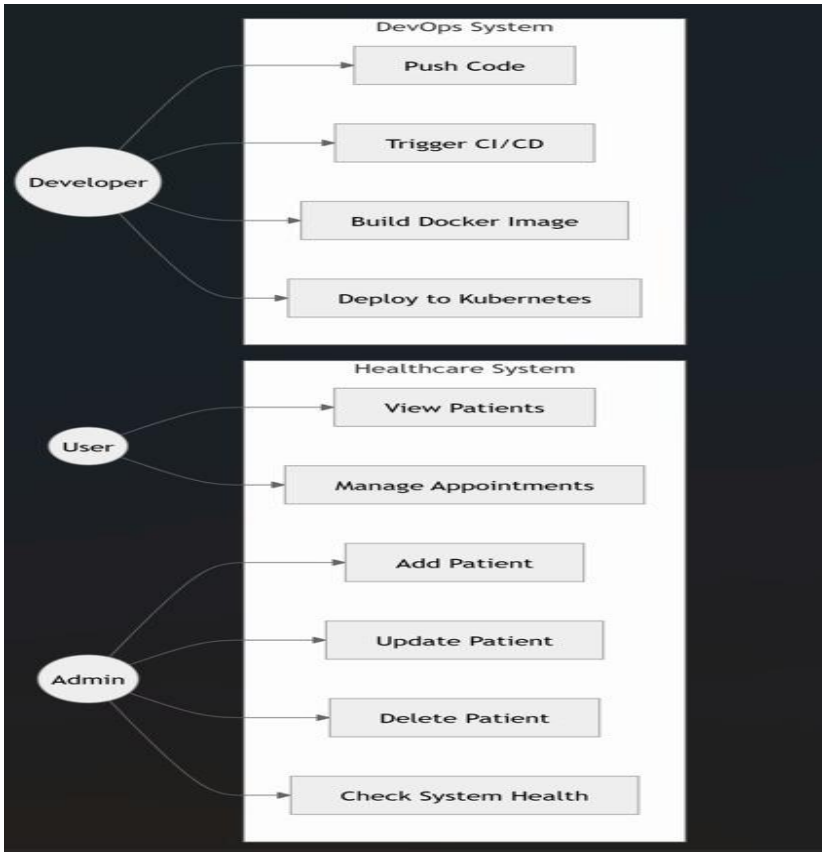

Figure 2 :Use Case Diagram

3.2 DATA ACCESS MECHANISM

1. Backend interacts with database using ORM (e.g., SQLAlchemy)
2. Secure credentials stored in environment variables
3. Database connection handled through Docker networking

3.3 DATA RETENTION POLICIES

1. Data is stored securely in the database
2. Logs are maintained for debugging
3. Backup strategies can be implemented

3.4 DATA MIGRATION

1. Database schema changes handled via migrations
2. New containers deploy updated versions
3. Rollback possible if issues occur

4. INTERFACES

4.1 USER INTERFACE

1. APIs accessed via tools like Postman or frontend apps
2. Docker Desktop for monitoring containers

4.2 COMMAND LINE INTERFACE

1. Docker CLI for building images
2. Docker Compose for running services

5. STATE AND SESSION MANAGEMENT

1. Backend remains stateless
2. Sessions handled using tokens (if required)
3. Database maintains persistent state

6. CACHING

1. Docker layer caching improves build performance
2. Dependencies cached for faster rebuilds

7. FUNCTIONAL REQUIREMENTS

7.1 AUTOMATED BUILD AND TESTING PROCESS

1. Application is built using Dockerfile
2. Dependencies installed automatically

7.2 APPLICATION CONTAINERIZATION USING DOCKER

1. Multi-stage builds reduce image size
2. Application runs consistently across environments

7.3 DEPLOYMENT ON AWS EC2

1. The system must deploy the containerized application on AWS EC2 instances.
2. The EC2 instance should pull the Docker image from a Docker registry.
3. The application must run inside a Docker container on the EC2 server.

7.4 MONITORING AND LOGGING

1. Logs generated by Docker
2. Used for debugging and monitoring

8. NON-FUNCTIONAL REQUIREMENTS

8.1 SECURITY

1. Secure environment variables
2. Minimal attack surface

8.2 PERFORMANCE

1. Faster builds and deployments
2. Efficient resource utilization

8.3 SCALABILITY

1. Easy container scaling
2. Supports growing user base

8.4 RELIABILITY

1. Consistent execution environment
2. Reduced deployment failures

9. REFERENCES

- Docker Official Documentation
- Flask Documentation
- Python Documentation
- Docker Compose Documentation

10. GITHUB

https://github.com/BokilAnimesh/Dockerized-Healthcare-Python-Flask-Service